

Tarea #1



Esteban Barrera Sanabria - esbarreras@unal.edu.co

Universidad Nacional de Colombia

Facultad de Ingeniería

Ingeniería de Software II

Sergio Enrique Vargas Pedraza

Febrero 2026



En el documento presente se realiza un resumen del video [Principios SOLID - Programación en español](#) de YouTube, donde se explica el concepto y la importancia del protocolo SOLID en el ambiente del clean code para software.

Los principios SOLID son un conjunto de cinco directrices fundamentales para el diseño de software especialmente orientado a objetos, cuyo objetivo principal es crear un software más legible, limpio y escalable; en consecuencia, al aplicarlos, se reducen las dependencias y se facilita la evolución del código a lo largo del tiempo.

Es igual de importante, según el autor, aprender a detectar el incumplimiento de estos estándares del clean code en código propio que implementarlos en sí, pues permite escalar a código de terceros.

S - Principio de Responsabilidad Única (SRP)

Este principio establece que una clase debe tener una sola razón para cambiar en el tiempo, es decir, debe encargarse de una única funcionalidad dentro del sistema; de esta manera, si un módulo realiza múltiples tareas, los cambios en una de ellas podrían afectar incorrectamente a las otras.

```
Python
# Sin SRP
class Usuario:
    def __init__(self, nombre, email):
        self.nombre = nombre
        self.email = email

    def enviar_correo_bienvenida(self):
        print(f"Enviando correo a {self.email}")

# Con SRP
class Usuario:
    def __init__(self, nombre, email):
        self.nombre = nombre
        self.email = email

class ServicioEmail:
    def enviar_bienvenida(self, usuario):
        print(f"Enviando correo a {usuario.email}")
```



En el ejemplo, en lugar de crear una clase Usuario que gestione tanto los datos del usuario como el envío de correos, se separan en dos clases distintas.

O - Principio de Abierto/Cerrado (Open/Closed Principle)

Este principio dicta que las entidades de software (clases, módulos, funciones) deben estar abiertas para su **extensión** pero cerradas para su **modificación**; en otras palabras, se debe poder añadir nuevo comportamiento sin alterar el código fuente existente, esto gracias a conceptos de POO como la abstracción y la herencia.

```
Python
from abc import ABC, abstractmethod

class Forma(ABC):
    @abstractmethod
    def area(self):
        pass

class Rectangulo(Forma):
    def __init__(self, ancho, alto):
        self.ancho = ancho
        self.alto = alto
    def area(self):
        return self.ancho * self.alto

# Usando Open/Closed
class Circulo(Forma):
    def __init__(self, radio):
        self.radio = radio
    def area(self):
        return 3.14 * self.radio ** 2

def area_total(formas):
    return sum(forma.area() for forma in formas)
```

En el ejemplo, se define una clase base abstracta para Forma y se extiende para nuevas formas sin modificar el código que calcula el área total.

L - Principio de Sustitución de Liskov (Liskov Substitution Principle)

Este principio indica que las clases derivadas deben ser sustituibles por sus clases base sin alterar el correcto funcionamiento del programa; es decir, si una subclase no puede realizar las mismas acciones que su clase padre, entonces no se trata de una verdadera herencia y se debe reconsiderar el diseño.



Python

```
class Pato:
    def nada(self):
        return "El pato está nadando"

class PatoVolador(Pato):
    def vuela(self):
        return "El pato está volando"

class PatoDeGoma(Pato):
    def nada(self):
        return "El pato de goma flota"
```

Una subclase PatoDeGoma no debe heredar de Pato si no puede volar, ya que violaría la sustitución; es mejor separar los comportamientos en interfaces específicas.

I - Principio de Segregación de Interfaces (Interface Segregation Principle)

Este principio aconseja que ninguna clase deba depender de métodos que no utiliza; en consecuencia, es preferible contar con interfaces pequeñas y específicas en lugar de una interfaz general grande, esto ayuda a que las implementaciones no definan comportamientos innecesarios.

Python

```
from abc import ABC, abstractmethod

class Trabajador(ABC):
    @abstractmethod
    def trabajar(self):
        pass

class Comedor(ABC):
    @abstractmethod
    def comer(self):
        pass

class Humano(Trabajador, Comedor):
    def trabajar(self):
        print("Humano trabajando")
    def comer(self):
        print("Humano comiendo")
```



```
class Robot(Trabajador):  
    def trabajar(self):  
        print("Robot trabajando")
```

En el ejemplo, en lugar de una interfaz Trabajador con métodos trabajar() y comer(), se separan en dos interfaces para que un robot pueda trabajar sin tener que implementar el método comer.

D - Principio de Inversión de Dependencias (Dependency Inversion Principle)

Finalmente, este principio sostiene que los módulos de alto nivel no deben depender de módulos de bajo nivel, sino que ambos deben depender de abstracciones.

Asimismo, las abstracciones no deben depender de los detalles, sino los detalles de las abstracciones, lo que promueve un acoplamiento más flexible y facilita las pruebas unitarias.

```
Python  
from abc import ABC, abstractmethod  
  
class BaseDatos(ABC):  
    @abstractmethod  
    def guardar(self, datos):  
        pass  
  
class BaseDatosMySQL(BaseDatos):  
    def guardar(self, datos):  
        print(f"Guardando {datos} en MySQL")  
  
class GestorUsuario:  
    def __init__(self, base_datos: BaseDatos): # Dependencia  
        (abstracción)  
        self.base_datos = base_datos  
  
    def registrar(self, usuario):  
        self.base_datos.guardar(usuario)  
  
# Uso  
db = BaseDatosMySQL()  
gestor = GestorUsuario(db)  
gestor.registrar("Juan Perez")
```



Aunque puede verse confuso, en el ejemplo se ilustra que se inyecta una dependencia abstracta (base de datos) en lugar de que la clase GestorUsuario cree una instancia concreta de una base de datos específica.

Esto nos permite ver que los principios SOLID no son solo estándares de escalabilidad y legibilidad sino robustez y adaptabilidad. También, como se puede notar, forman una amalgama de consejos derivados de los conceptos de la programación orientada a objetos, desde clases, pasando por abstracciones y terminando en dependencias.